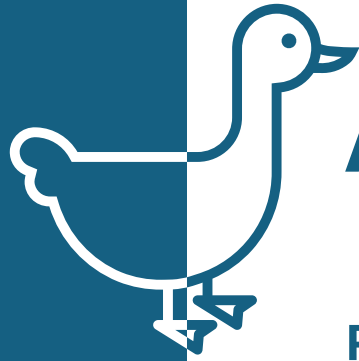


STL

und die
verlorene Ente



AI for Business Prognosis 2026

Präsentiert von:

Jasmin Stribik
Fabian Schmid
Islam Abdalla

Beobachtete Reihe y_t
(Ente im Meer)



Trend T_t (Gezeiten)

Langfristige Veränderung des
Wasserspiegels (z. B. Ebbe und Flut).



+

Saisonale Komponente S_t (Wellen)

Regelmäßige, wiederkehrende Muster
(Wellen kommen und gehen).



+

Residuum R_t (Jet-Ski)

Unregelmäßige, zufällige Einflüsse
(z. B. ein Tourist mit Jet-Ski).



Zusammengefasst:

$$y_t = T_t + S_t + R_t$$

(Ente im Meer = Gezeiten + Wellen + Jet-Ski)

Agenda

- 1 STL! – Was?
- 2 STL – Warum?
- 3 STL – Wie?
- 4 Robust STL
- 5 STL-Anwendungsfälle
- 6 Fazit

Agenda

1

STL! – Was?

2

STL – Warum?

3

STL – Wie?

4

Robust STL

5

STL-Anwendungsfälle

6

Fazit

STL! Was?

Was bedeutet STL?



Hinweise

STL zerlegt eine Zeitreihe in ihre bestandteile Trend, Saison und Residuum mithilfe von LOESS (lokaler Regression).

Wichtig: STL wird als Vorbereitung für andere Prognosemodelle verwendet und ist kein eigenständiger Forecasting-Algorithmus.

Das Zerlegungsmodell von STL

$$Y_t = T_t + S_t + R_t$$

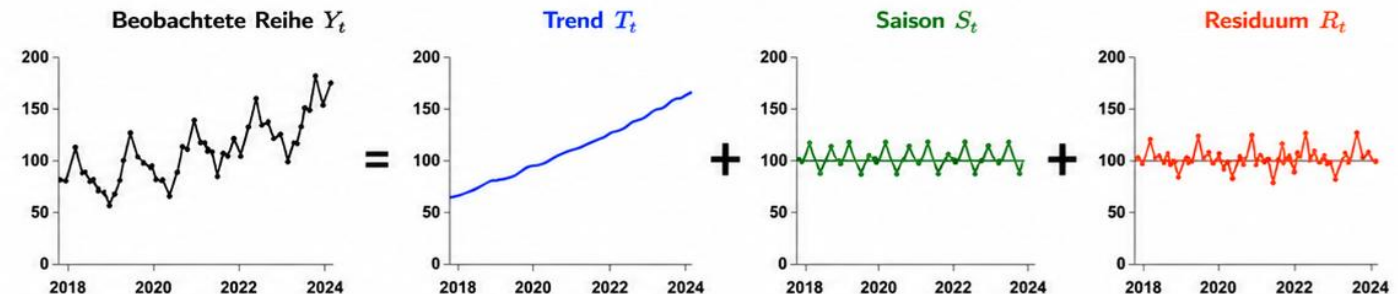
wobei:

Y_t : beobachteter Wert zum Zeitpunkt t

T_t : Trendkomponente (langfristige Entwicklung)

S_t : Saisonale Komponente (periodische Muster)

R_t : Residuum (Zufall / unregelmäßige Schwankungen)



Beobachtung = Trend + Saison + Residuum

STL - Historie

Historischer Hintergrund



1990

STL wurde 1990 von den Statistikern entwickelt:

- Robert Cleveland
- William Cleveland
- Jean McRae
- Irma Terpenning

Originalpublikation:
"STL: A Seasonal-Trend
Decomposition Procedure
Based on LOESS"
(1990)

Ziel der Entwicklung



Ein flexibles, robustes und intuitiv verständliches Verfahren zur Zerlegung von Zeitreihen, das die Einschränkungen klassischer Methoden überwindet.

Meilensteine



1990

Veröffentlichung des
STL-Papiers
(Cleveland et al.)



1991–1993

Verbreitung und erste
Anwendungen in Praxis
und Software



1997

Integration in S-PLUS
(später in R) als
Funktion `stl()`



2000–heute

Weiterentwicklungen, robuste
Varianten und Erweiterungen
(z. B. MSTL)

Agenda

1 STL! – Was?

2 STL – Warum?

3 STL – Wie?

4 Robust STL

5 STL-Anwendungsfälle

6 Fazit

STL – Warum?

Einschränkungen bei den klassischen Verfahren



Starre Annahmen

Konstante Saisonmuster und linearer Trend werden vorausgesetzt.



Nicht flexibel

Anpassung an lokale Strukturen oder komplexe Entwicklungen ist begrenzt.



Ausreißerempfindlich

Extreme Werte können die Schätzung von Trend und Saison stark verzerren.



Veraltete Methoden

Viele klassische Verfahren sind nicht für moderne, komplexe Zeitreihen geeignet.



Wenig Anpassungsfähigkeit

Schwierigkeiten bei sich ändernden Saisonlängen und nichtlinearen Trends.

Die Innovationen von STL



Flexibler Trend

LOESS-Glättung erfasst nichtlineare und lokale Trendverläufe.



Flexible Saison

Saisonmuster können sich über die Zeit verändern (zeitvariante Saisonalität).



Robust gegenüber Ausreißern

Robuste LOESS-Schätzung reduziert den Einfluss von Ausreißern.



Adaptiv und datengetrieben

Passt sich automatisch an die Struktur der Zeitreihe an – ohne starre Annahmen.



Breite Anwendbarkeit

Funktioniert für viele Arten von Zeitreihen aus verschiedenen Anwendungsbereichen.



STL bietet eine **flexible, robuste und intuitive Lösung** zur Zerlegung moderner Zeitreihen und überwindet die wesentlichen Grenzen klassischer Verfahren.

Agenda

- 1 STL! – Was?
- 2 STL – Warum?
- 3 STL – Wie?
- 4 Robust STL
- 5 STL-Anwendungsfälle
- 6 Fazit

STL – Wie?

STL-Algorithmus (iterativer Prozess)

1 Initialer Trend

Glätte Y_t mit LOESS $\rightarrow$ initialer Trend $T_t^{(0)}$

$$T_t^{(0)} = LOESS(Y_t)$$



2 Detrend

Entferne Trend: $D_t = Y_t - T_t$

$$D_t = Y_t - T_t$$



3 Saison-Schätzung

Für jede Saison-Subreihe (z.B. jeden Monat)
LOESS glätten $\rightarrow$ saisonale Komponente S_t

$$S_t = \text{LOESS pro Saison-Subreihe}$$



4 Entferne Saison

$Y_t^* = Y_t - S_t$ (nur Trend + Rauschen bleiben)

$$Y_t^* = Y_t - S_t$$



5 Trend aktualisieren

Glätte Y_t^* mit LOESS $\rightarrow$ neuer Trend T_t

$$T_t = LOESS(Y_t^*)$$



6 Residuum berechnen

$R_t = Y_t - T_t - S_t$

$$R_t = Y_t - T_t - S_t$$

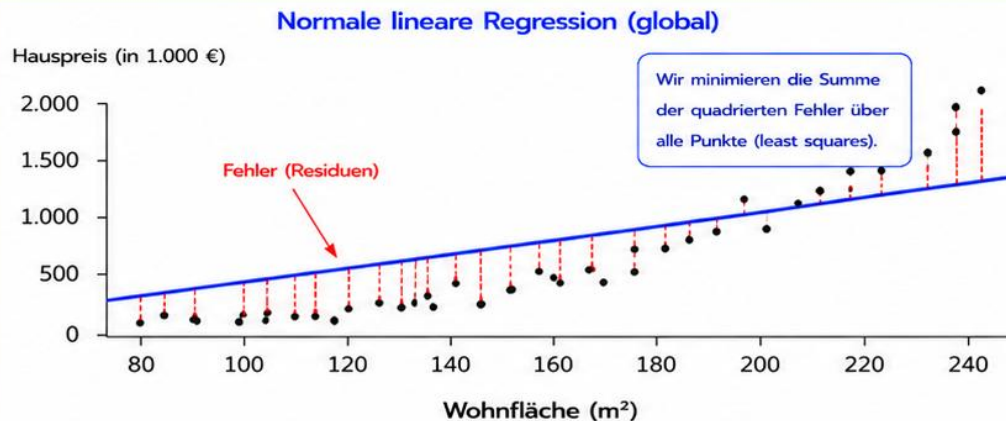
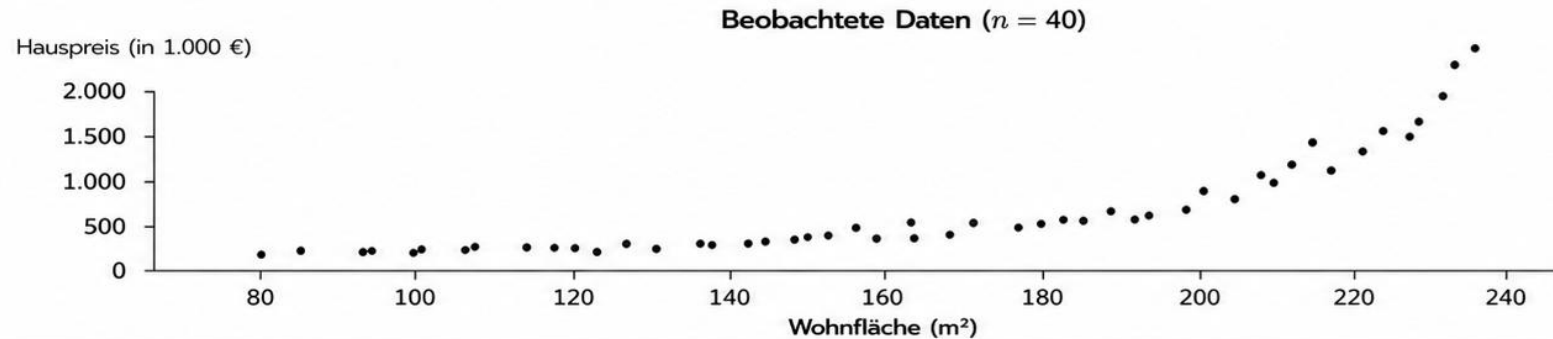


Wiederhole
Schritte 2–6
bis zur
Konvergenz
(typisch
5–20
Iterationen)

$$\text{Ergebnis: } Y_t = T_t + S_t + R_t$$

LOESS – Wie?

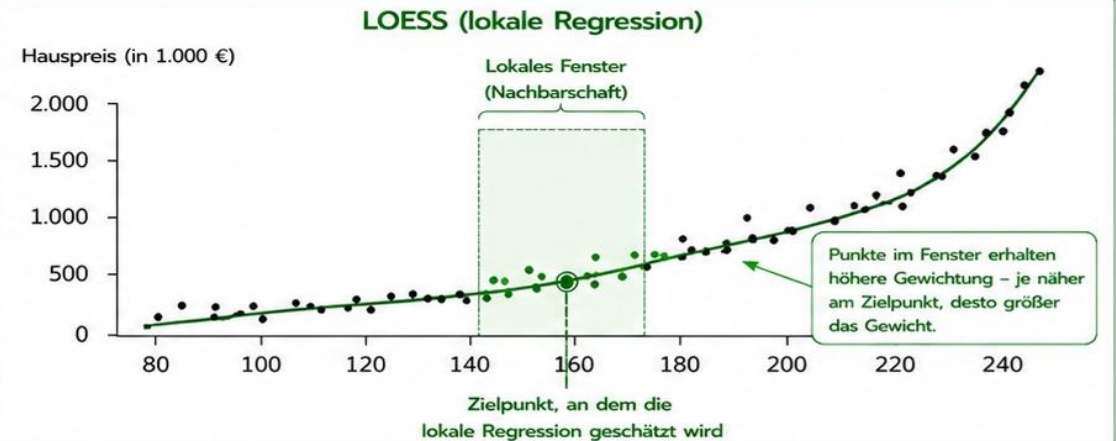
LOESS – Lokale (gewichtete) Regression



Lineare Regression (global):

Eine einzige Gerade für alle Daten.

Wir minimieren die Summe der quadrierten Fehler über alle Punkte.



LOESS (lokal):

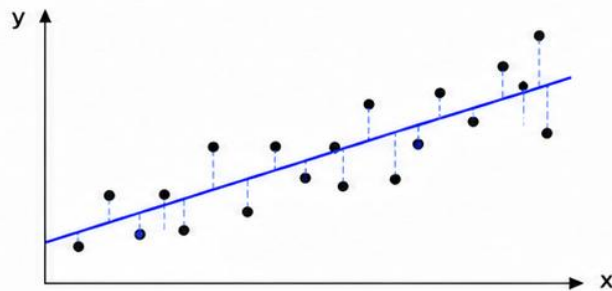
Lokale Anpassung an die Datenstruktur.

Im lokalen Fenster werden nahe Punkte höher gewichtet.

LOESS – Wie?

LOESS – Lokale (gewichtete) Regression

1. Normale lineare Regression (global)



Ziel:

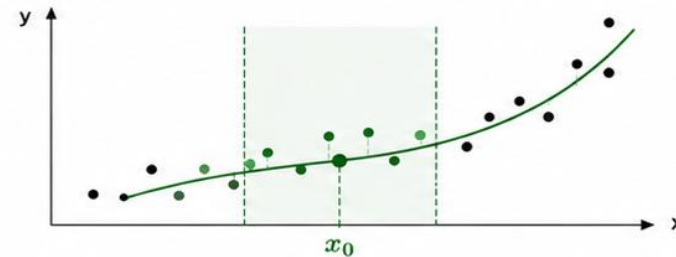
Minimiere die Summe der quadrierten Residuen (Least Squares).

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \min_{\beta_0, \beta_1} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Modell: $\hat{y}_i = \beta_0 + \beta_1 x_i$

Eine einzige Gerade für alle Datenpunkte.
Jeder Punkt trägt gleiches Gewicht.

2. LOESS (lokale gewichtete Regression)



Ziel:

Minimiere die Summe der gewichteten quadrierten Residuen (Weighted LS).
Nahe Punkte erhalten höheres Gewicht.

$$\min_{\beta_0(x_0), \beta_1(x_0)} \sum_{i=1}^n w_i(x_0) (y_i - \hat{y}_i(x_0))^2 = \min_{\beta_0(x_0), \beta_1(x_0)} \sum_{i=1}^n w_i(x_0) (y_i - \beta_0(x_0) - \beta_1(x_0)x_i)^2$$

Lokales Modell (bei x_0):

$$\hat{y}(x_0) = \beta_0(x_0) + \beta_1(x_0)x_0$$

Lokale Anpassung in einem Nachbarschaftsfenster um x_0 .

Nahe Punkte (dunkelgrün) haben mehr Gewicht als entfernte (hellgrün).

Gewichtungsfunktion (Beispiel: Tricube)

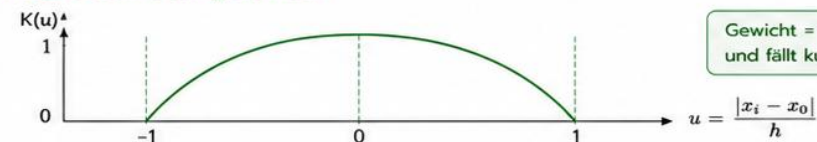
$$w_i(x_0) = K\left(\frac{|x_i - x_0|}{h}\right) \quad \text{mit} \quad K(u) = \begin{cases} (1 - |u|^3)^3, & |u| < 1 \\ 0, & |u| \geq 1 \end{cases}$$

x_0 : Zielpunkt (Ort der lokalen Anpassung)

h : Fensterbreite (Bandbreite)

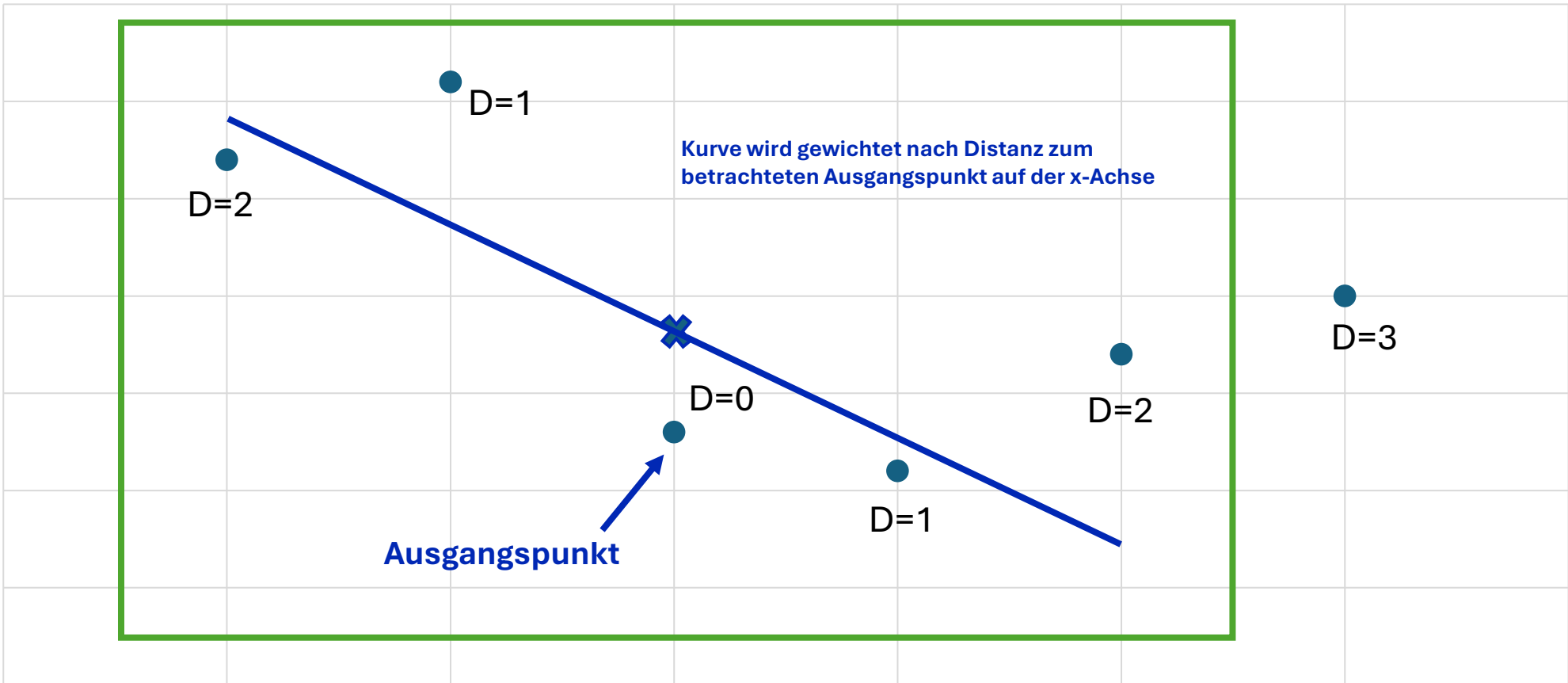
$$u = \frac{|x_i - x_0|}{h} : \text{normierte Distanz}$$

Tricube-Gewichtungsfunktion

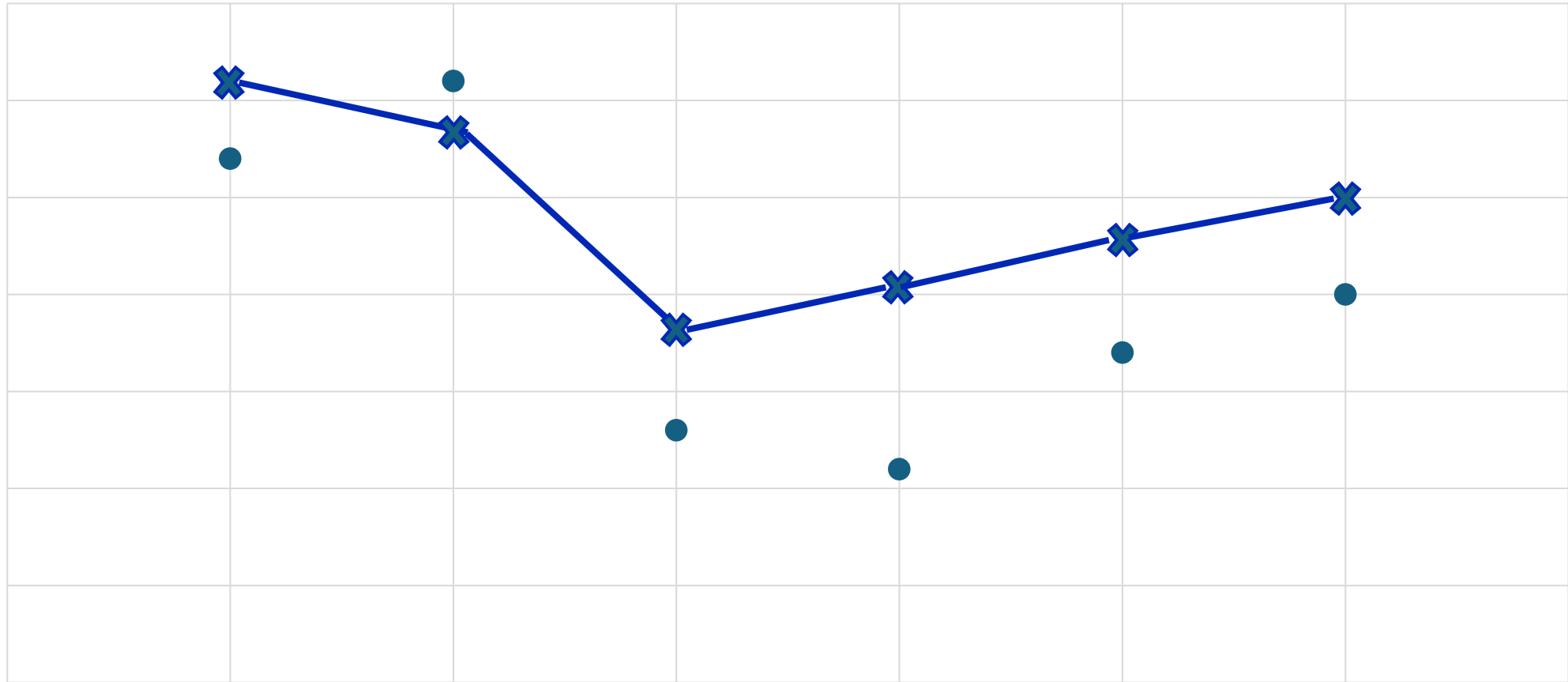


Gewicht = 1 am Zielpunkt ($u=0$)
und fällt kubisch auf 0 bei $|u| \geq 1$.

window-size = 5



Resultierende Glättung



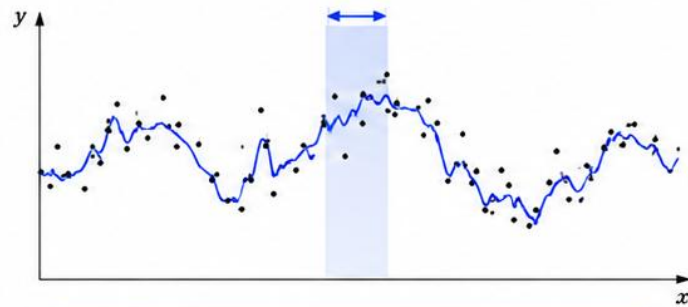
LOESS – Wie?

LOESS: Einfluss der Fenstergröße (Span)

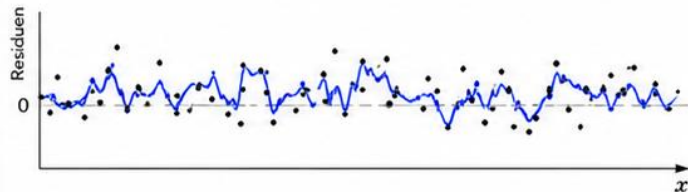


Die Fenstergröße (Span) bestimmt, wie viele Nachbarpunkte für die lokale Anpassung verwendet werden.

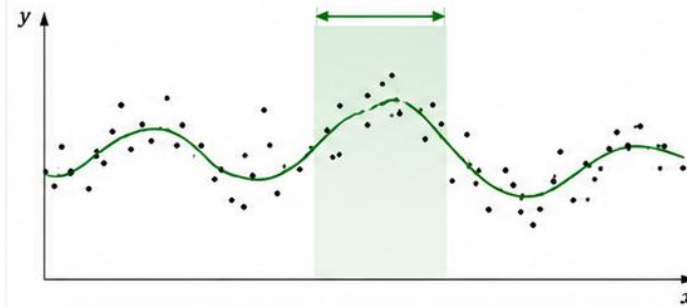
Kleine Fenstergröße (kleiner Span)



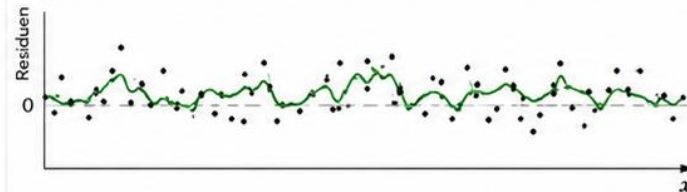
Folgt den Daten sehr genau
Hohe Varianz (überanpassungsanfällig)



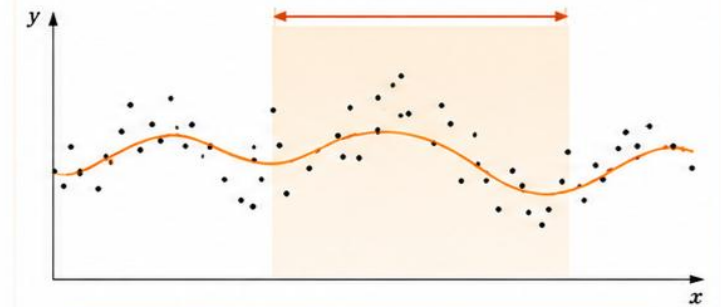
Mittlere Fenstergröße (moderater Span)



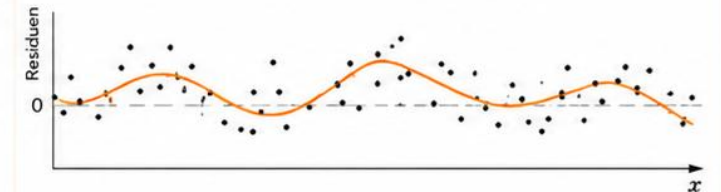
Guter Kompromiss
Ausgewogen zwischen Bias und Varianz



Große Fenstergröße (großer Span)



Sehr glatte Anpassung
Hoher Bias (unteranpassungsanfällig)



Kernaussage: Kleine Fenster $\rightarrow$ flexibel, aber rauschanfällig.

Große Fenster $\rightarrow$ stabil, aber zu glatt.

Die Wahl des Spans ist entscheidend!

Agenda

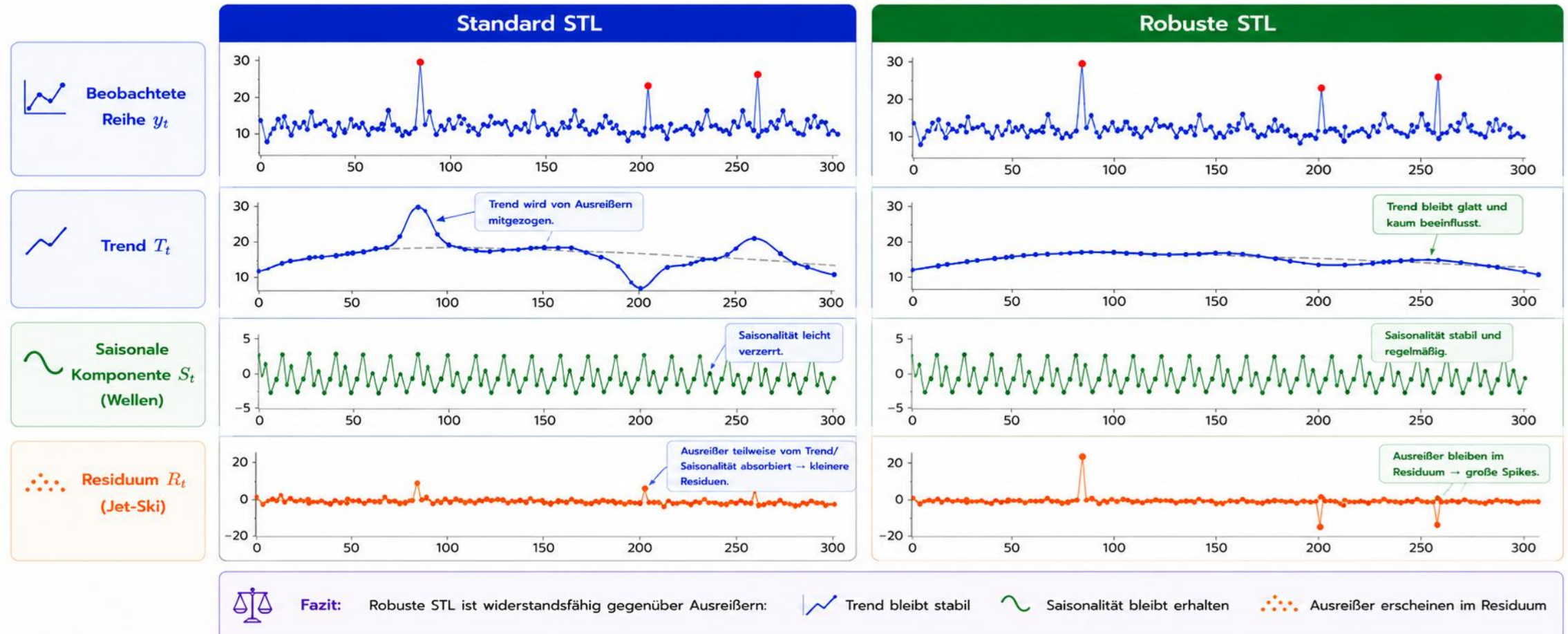
- 1 STL! – Was?
- 2 STL – Warum?
- 3 STL – Wie?
- 4 Robust STL
- 5 STL-Anwendungsfälle
- 6 Fazit

Standard vs. Robust STL

Vergleich: Standard STL vs. Robuste STL

Gleiche beobachtete Reihe mit Ausreißern (rote Punkte).

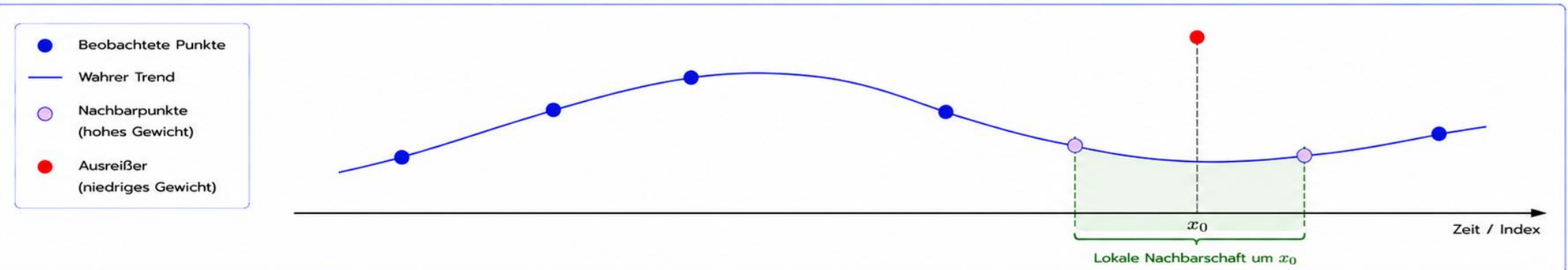
● Ausreißer (extreme Punkte)



Robust STL

Robustes LOESS – Mathematik

LOESS schätzt den lokalen Trend in der Umgebung jedes Punktes. Robustes LOESS reduziert den Einfluss von Ausreißern (extremen Punkten) durch robuste Gewichte.



1 Allgemeine LOESS-Schätzung mit robuster Gewichtung

Für ein Zielpunkt x_0 wird der lokale Trend $\hat{m}(x_0)$ als gewichtete lineare Regression in der Nachbarschaft $\mathcal{N}(x_0)$ geschätzt:

$$\min_{\beta_0, \beta_1} \sum_{i \in \mathcal{N}(x_0)} \left(W_i^{(d)} \cdot W_i^{(r)} \right) (y_i - \beta_0 - \beta_1 (x_i - x_0))^2$$

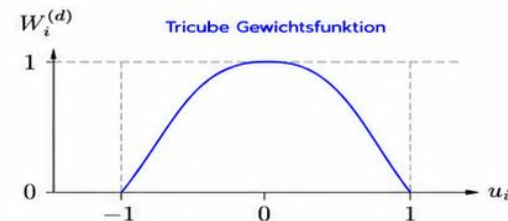
Entfernungsgewicht $W_i^{(d)}$ Robustgewicht $W_i^{(r)}$

y_i	Beobachtete Werte
x_i	Positionen (Zeit / Index)
$W_i^{(d)}$	Gewichte basierend auf der Entfernung von x_0 (z. B. Tricube)
$W_i^{(r)}$	Robustes Gewicht basierend auf Residuen (z. B. Tukey's Biweight)

2 Entfernungsgewicht: Tricube-Kern

Mit $u_i = \frac{|x_i - x_0|}{h}$ (h = Bandbreite)

$$W_i^{(d)} = \begin{cases} (1 - |u_i|^3)^3, & |u_i| < 1 \\ 0, & |u_i| \geq 1 \end{cases}$$

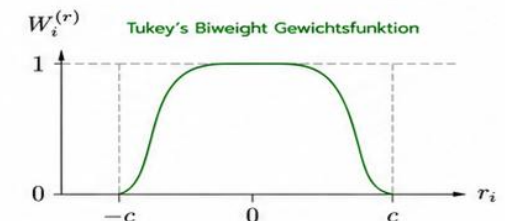


3 Robustes Gewicht: Tukey's Biweight

Mit $r_i = \frac{e_i}{s}$ (standardisierte Residuen, $s = MAD$)

$$W_i^{(r)} = \begin{cases} \left(1 - \left(\frac{r_i}{c}\right)^2\right)^2, & |r_i| < c \\ 0, & |r_i| \geq c \end{cases}$$

typisch $c = 4.685$



STL – Recap?

STL-Algorithmus (iterativer Prozess)

1 Initialer Trend

Glätte Y_t mit LOESS $\rightarrow$ initialer Trend $T_t^{(0)}$

$$T_t^{(0)} = LOESS(Y_t)$$



2 Detrend

Entferne Trend: $D_t = Y_t - T_t$

$$D_t = Y_t - T_t$$



3 Saison-Schätzung

Für jede Saison-Subreihe (z.B. jeden Monat)
LOESS glätten $\rightarrow$ saisonale Komponente S_t

$$S_t = LOESS \text{ pro Saison-Subreihe}$$



4 Entferne Saison

$Y_t^* = Y_t - S_t$ (nur Trend + Rauschen bleiben)

$$Y_t^* = Y_t - S_t$$



5 Trend aktualisieren

Glätte Y_t^* mit LOESS $\rightarrow$ neuer Trend T_t

$$T_t = LOESS(Y_t^*)$$



6 Residuum berechnen

$R_t = Y_t - T_t - S_t$

$$R_t = Y_t - T_t - S_t$$



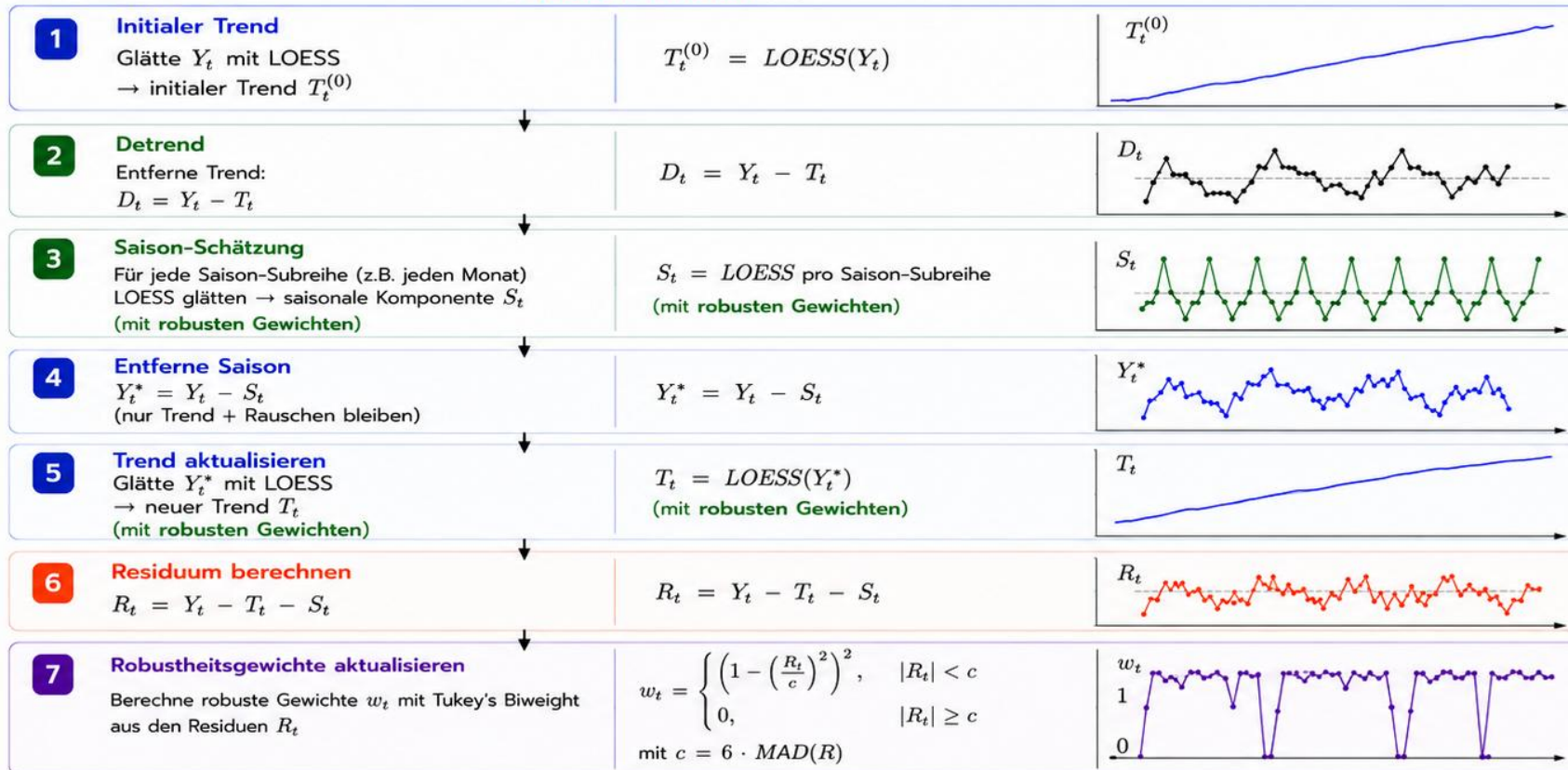
Wiederhole
Schritte 2–6
bis zur
Konvergenz
(typisch
5–20
Iterationen)

$$\text{Ergebnis: } Y_t = T_t + S_t + R_t$$

Robust STL

Robust STL – Iterativer Algorithmus (Schritte 1–7)

Robust STL (Seasonal–Trend decomposition using LOESS) zerlegt eine Zeitreihe y_t in Trend T_t , saisonale Komponente S_t und Residuum R_t durch einen iterativen Prozess mit LOESS und **robusten Gewichten** (gegen Ausreißer).



Innere Schleife
(Wiederhole
Schritte 2–6
bis zur
Konvergenz*)
(typisch
5–20
Iterationen)

Äußere Schleife
(Wiederhole
Schritte 2–7
bis zur
Konvergenz**
der Gewichte)
(typisch
2–10
Iterationen)

Ergebnis: $Y_t = T_t + S_t + R_t$

* Konvergenz, wenn Änderungen in T_t , S_t und R_t unter einer Toleranz liegen.

** Konvergenz, wenn Änderungen der Gewichte w_t unter einer Toleranz liegen.

Agenda

- 1 STL! – Was?
- 2 STL – Warum?
- 3 STL – Wie?
- 4 Robust STL
- 5 STL-Anwendungsfälle
- 6 Fazit

STL - Anwendungsfälle

STL – Wann funktioniert es und wann nicht?

STL funktioniert gut bei stabilen, additiven Strukturen – und weniger gut bei starken strukturellen Brüchen oder nicht-additiven Mustern.

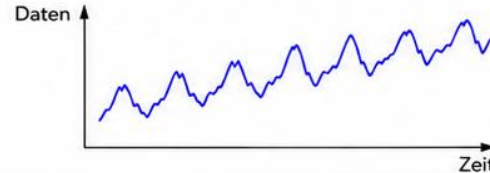
✓ STL funktioniert gut bei ...



1 Stabile saisonale Muster

Konstante Saisonalität und Trend über die Zeit.

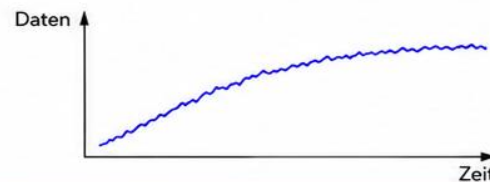
Beispiel: Einzelhandelsumsatz



2 Langsame Trendänderungen

Graduale Änderungen im Trend (z. B. Wachstum, Sättigung).

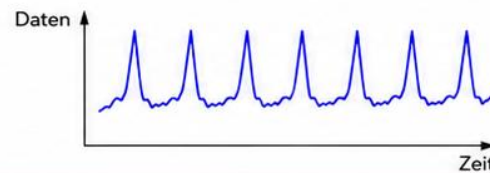
Beispiel: Abonnentenwachstum



3 Feste Kalender-Saisonalität

Wöchentliche / monatliche Muster mit konstanter Amplitude.

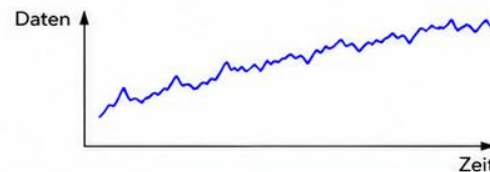
Beispiel: Tourismusnachfrage



4 Iterative Prognose / Planung

Für rollierende Forecasts und Regel-Updates geeignet.

Beispiel: Energieverbrauch



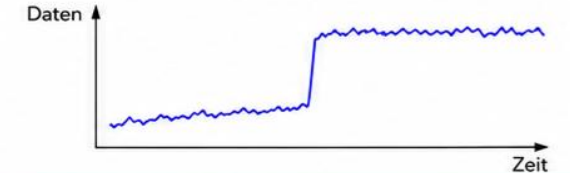
✗ STL funktioniert weniger gut bei ...



1 Strukturelle Brüche

Plötzliche Niveauwechsel oder Trendbrüche.

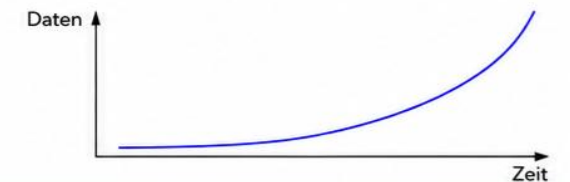
Beispiel: Pandemiebreak



2 Nicht-lineare / exponentielle Trends

Stark gekrümmte oder exponentielle Entwicklungen.

Beispiel: Nutzerwachstum



3 Zeitlich wechselnde Saisonalität

Ändernde Amplitude, Frequenz oder unregelmäßige Muster.

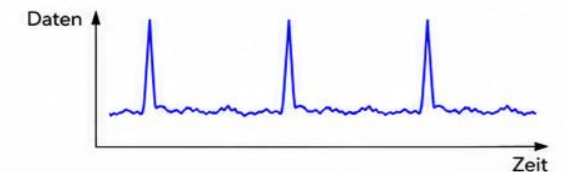
Beispiel: Werbekampagnen



4 Promo- / Eventeffekte

Starke, kurzzeitige Ausreißer durch Aktionen oder Events.

Beispiel: Black Friday



Agenda

- 1 STL! – Was?
- 2 STL – Warum?
- 3 STL – Wie?
- 4 Robust STL
- 5 STL-Anwendungsfälle
- 6 Fazit

Fazit

- **STL zerlegt Zeitreihen in Trend, Saison und Residuum** – iterativ über LOESS-Glättung.
- Flexibel steuerbar über Window-Size und Glättungsgrad; bewältigt sich **ändernde Saisonalität**.
- **Robust STL** liefert stabile, verlässliche Komponenten, da Ausreißer geringere Gewichte erhalten.
- **Praxis:**
 - ideal für Nachfrageprognosen bei Kombination mit einem anderen Modell
 - Gut geeignet für additive Zeitreihen
 - Nicht geeignet für sehr kurze oder nicht-saisonale Reihen z.B. Aktien.



Audience Q&A

① The Slido app must be installed on every computer you're presenting from



Welche Komponenten werden durch STL geschätzt?



**Welche Komponente enthält bei
Robust STL typischerweise die
Ausreißer?**

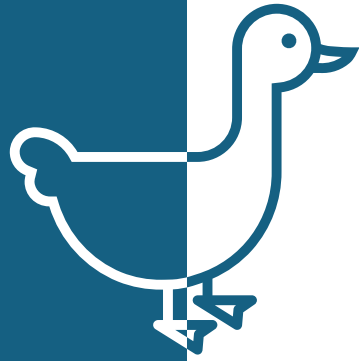


**Welche Anwendungsfall ist für STL
gut geeignet?**

Quellen

- Bandara, Kasun, Rob Hyndman, und Christoph Bergmeir. „MSTL: A Seasonal-Trend Decomposition Algorithm for Time Series with Multiple Seasonal Patterns“. International Journal of Operational Research 1, Nr. 1 (2022): 1. <https://doi.org/10.1504/IJOR.2022.10048281>.
- Haller, Vincent, Sebastien Daniel, und Benoit Bellone. „STAHL: Seasonal, Trend, and Holiday Decomposition with Loess“. Journal of Official Statistics 41, Nr. 4 (2025): 1112–53. <https://doi.org/10.1177/0282423X251344219>.
- OTexts. Forecasting Principles & Practice: 3.6 STL decomposition. 2023. 10:56. <https://otexts.com/fpppy/03-decomposition.html#sec-stl>.
- OTexts. Forecasting Principles & Practice: 3.6 STL decomposition. 2023. <https://www.youtube.com/watch?v=Q3JvdEiqv-4>.
- Robert B. Cleveland, William S. Cleveland, Jean E. McRae, and Irma Terpenning. „STL: A Seasonal-Trend Decomposition Procedure Based on Loess“. Journal of Official Statistics 6, Nr. 1 (1990): 3–73.
- Starmer, Josh. The Statquest Illustrated Guide to Machine Learning!!! Selbstverlag, 2022, ISBN: 978-1-80461-891-2.

Danke!



Fragen?

Vielen Dank für Ihre Aufmerksamkeit!
Wir freuen uns auf Ihre Fragen und die
Diskussion.